

RESUMEN REPOSITORIOS

Repositorios: Estructura y Actualización	2
Definición Técnica Literal.....	2
Tipos de Repositorios	2
Conceptos en Sistemas Distribuidos (Git)	2
Sistemas Centralizados (SVN) vs Distribuidos (Git)	3
Generación de Código y Documentación	3
Herramientas de Generación.....	3
Documentación del Proyecto	3
Metodologías de Desarrollo	4
MÉTRICA v3: Los 5 Procesos e Interfaces Clave	4
Tipos de Mantenimiento en MÉTRICA v3 (MSI): Correctivo, Evolutivo, Adaptativo, Perfectivo	4
Refactorización: Concepto y relación con el mantenimiento perfectivo.....	5
Metodologías Ágiles (Manifiesto Ágil y SCRUM)	5
Kanban: Flujo continuo y limitación del WIP	5
SCRUM: Roles, Artefactos y Ceremonias.....	5
Estimación en SCRUM: Story Points y Planning Poker	6
Calidad del Software: La norma ISO/IEC 25010.....	6
Pruebas de Software	6
Tipos de Pruebas según MÉTRICA v3	6
Técnicas: Caja Blanca vs Caja Negra	7
Técnicas y Herramientas Avanzadas.....	7
Herramientas: Selenium (Funcional), JMeter (Carga) y Cucumber (BDD).....	7
TDD (Test Driven Development): El ciclo Red-Green-Refactor.	8
Pruebas de Regresión y Pruebas de Humo (Smoke Tests)	8
Métricas de Calidad: La Complejidad Ciclomática de McCabe	8
Programas para Control de Versiones	9
Clasificación	9

Estados de los Archivos en Git	9
Comandos de Git	10
Plataformas de Desarrollo Colaborativo	10

Repositorios: Estructura y Actualización

Definición Técnica Literal

- **Repositorio:** "Sitio centralizado donde se almacena y mantiene información digital (bases de datos, archivos, imágenes, vídeo) preparada para distribuirse a través de un medio físico o una red informática. Engloba un conjunto de servicios para gestionar y difundir los materiales digitales producidos a través del autoarchivo".

Tipos de Repositorios

- **Repositorios de recursos:** "Contienen tanto los objetos con su contenido como los metadatos".
- **Repositorios de aprendizaje:** "Basan su contenido en unidades de aprendizaje que tienen como principal objetivo transmitir un conocimiento concreto".
- **Repositorios institucionales:** "Incluyen material académico diverso, tendiendo a ser organizados por una institución más que por áreas temáticas".

Conceptos en Sistemas Distribuidos (Git)

- **Repositorio Bare:** "Un repositorio bare se suele usar para crear un repositorio central de Git remoto".

Enfoque CPS: El tribunal suele preguntar por la función del "repositorio bare" como nodo central sin copia de trabajo. Otro distractor común es confundir los tipos de repositorios (institucionales vs temáticos).

Sistemas Centralizados (SVN) vs Distribuidos (Git)

- **Centralizados (CVCS):** Existe un único servidor con todos los archivos y los clientes hacen "checkout" de archivos individuales (ej. Subversion/SVN, CVS). Si el servidor cae, se pierde el historial si no hay backup.
- **Distribuidos (DVCS):** Cada cliente tiene una copia completa (un clon) del repositorio. Permite trabajar offline y es más seguro (ej. Git, Mercurial).

Generación de Código y Documentación

Herramientas de Generación

- **Generador de Código:** "Permite agilizar el desarrollo de aplicaciones mediante la generación automática de código a partir de diagramas o modelos. Numerosas herramientas CASE permiten la generación automática".
- **Javadoc:** Herramienta que "se podría utilizar para generar automáticamente la documentación de la aplicación a partir del código fuente" en tecnología Java.

Documentación del Proyecto

- **Fichero README.MD:** "Fichero en el que se incluirá, entre otra información, el propósito del proyecto y cómo arrancarlo localmente".
- **Manuales de Usuario:** Según MÉTRICA v3, los participantes en la tarea de "Elaboración de los Manuales de Usuario" son el Equipo de Proyecto.
- **Estándares:** Doxygen es un estándar/herramienta para la documentación de software. Otros estándares relacionados son ISO/IEC/IEEE 26511:2018.

Enfoque CPS: Una "trampa" recurrente es el nombre exacto de los ficheros de documentación en Git (ej. README.MD vs INFO.TXT). También se pregunta por herramientas de análisis de calidad como SonarQube.

Metodologías de Desarrollo

MÉTRICA v3: Los 5 Procesos e Interfaces Clave

- **Definición:** "Ofrece a las Organizaciones un instrumento útil para la sistematización de las actividades que dan soporte al ciclo de vida del software".
- **Procesos de Desarrollo:**
 - **EVS: Estudio de Viabilidad del Sistema.**
 - **ASI: Análisis del Sistema de Información.**
 - **DSI: Diseño del Sistema de Información.**
 - **CSI: Construcción del Sistema de Información.**
 - **IAS: Implantación y Aceptación del Sistema.**



Tipos de Mantenimiento en MÉTRICA v3 (MSI): Correctivo, Evolutivo, Adaptativo, Perfectivo

Dentro del proceso MSI (Mantenimiento de Sistemas de Información), es obligatorio distinguir por qué se cambia el software:

- **Correctivo:** Para corregir errores (bugs) encontrados por los usuarios.
- **Evolutivo:** Para añadir nuevas funcionalidades solicitadas por el cliente.
- **Adaptativo:** Para adaptar el sistema a cambios en el entorno (ej. un cambio de versión del SO o de la base de datos).
- **Perfectivo:** Para mejorar la eficiencia, el rendimiento o la mantenibilidad sin cambiar la funcionalidad.

Refactorización: Concepto y relación con el mantenimiento perfecto

- **Definición:** "Proceso de reestructurar el código fuente, alterando su estructura interna sin cambiar su comportamiento externo".
- **Objetivo:** Mejorar la legibilidad, reducir la complejidad y facilitar el mantenimiento futuro (coincide con el mantenimiento perfecto).

Metodologías Ágiles (Manifiesto Ágil y SCRUM)

- **Principio del Manifiesto Ágil:** "Son bienvenidos los requisitos cambiantes, incluso si llegan tarde al desarrollo".
- **Sprint Retrospective:** Reunión de SCRUM donde se comentan "malas prácticas para buscar la mejora continua y evitar aplicarlas en los siguientes sprints".

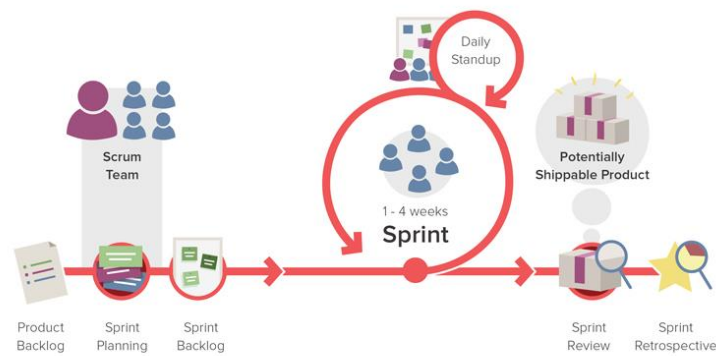
Enfoque CPS: En MÉTRICA v3, CPS suele preguntar por la Actividad EVS 1.2 (Identificación del Alcance), donde participan el Comité de Dirección, el Jefe de Proyecto y los Analistas. En metodologías ágiles, el foco es la prioridad del software funcionando sobre la documentación.

Kanban: Flujo continuo y limitación del WIP

- **Kanban:** No tiene iteraciones fijas (sprints), se basa en el flujo continuo y en limitar el WIP (Work In Progress) para evitar cuellos de botella. Usa un tablero visual.

SCRUM: Roles, Artefactos y Ceremonias

- **Roles:** Product Owner (voz del cliente), Scrum Master (facilitador), Developers (quienes ejecutan).
- **Artefactos:** Product Backlog (lista total de deseos), Sprint Backlog (tareas del sprint actual), Incremento (resultado de valor al final del sprint).
- **Ceremonias:** Sprint Planning, Daily Stand-up (15 min), Sprint Review, Retrospective.



Estimación en SCRUM: Story Points y Planning Poker

Al hablar de SCRUM, suelen preguntar cómo se mide el esfuerzo:

- **Story Points (Puntos de Historia):** Unidad de medida relativa (no en horas) que expresa la complejidad, el esfuerzo y el riesgo de una tarea.
- **Planning Poker:** Técnica basada en el consenso donde el equipo usa cartas (normalmente con la serie de Fibonacci: 1, 2, 3, 5, 8...) para estimar los puntos de historia.

Calidad del Software: La norma ISO/IEC 25010

- **ISO/IEC 25010:** Sustituye a la antigua ISO 9126. Define los 8 pilares de calidad: Adecuación funcional, Eficiencia, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad, Portabilidad y Compatibilidad.

Pruebas de Software

Tipos de Pruebas según MÉTRICA v3

- **Pruebas de Integración:** "Permiten verificar que los componentes o subsistemas interactúan correctamente a través de sus interfaces, tanto internas como externas, cubren la funcionalidad establecida y se ajustan a los requisitos".
- **Pruebas de Rendimiento:** Tienen como objetivo "determinar tiempos de respuesta dentro de los intervalos establecidos".

- **Pruebas de Regresión:** Su objetivo es "validar que un sistema cumple con el funcionamiento esperado" tras una modificación. Nota Crítica: "Sí es posible automatizar las pruebas de regresión" (el examen marca como falso que no se pueda).

Técnicas: Caja Blanca vs Caja Negra

- **Caja Blanca (Estructural):** Se conoce el código fuente. Se prueba la lógica interna (bucles, condiciones).
- **Caja Negra (Funcional):** No se mira el código, solo si para una entrada (input) la salida (output) es la esperada. Ej: Partición de equivalencia, Valores límite.

Técnicas y Herramientas Avanzadas

- **Fuzzing Testing:** "Conjunto de pruebas de caja negra que permiten descubrir errores de implementación mediante la introducción de datos al azar, inválidos o malformados".
- **TDD (Test-Driven Development):** "Práctica de programación en la que se diseñan las pruebas antes que el código fuente".
- **Herramientas:** JMeter (Carga y estrés), Selenium (Pruebas de aplicaciones web), Cucumber (BDD - Behavior Driven Development).

Enfoque CPS: Es vital distinguir entre pruebas unitarias (un componente) e integración (interfaces entre ellos). CPS pregunta frecuentemente por JMeter en escenarios de estrés y concurrencia.

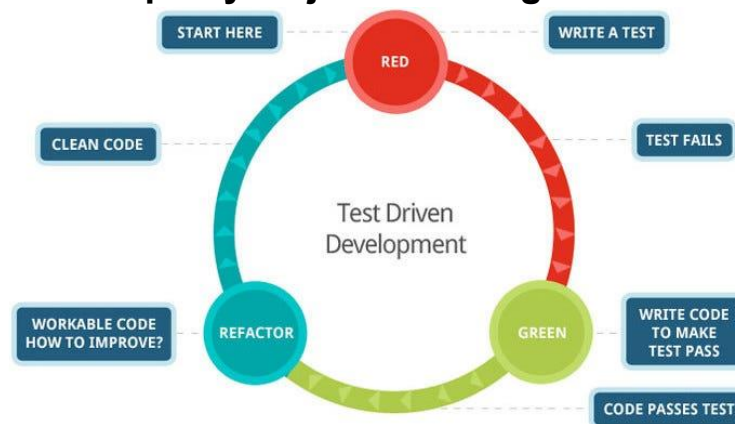
Herramientas: Selenium (Funcional), JMeter (Carga) y Cucumber (BDD)

- **Selenium:** Herramienta para automatizar pruebas funcionales de navegador (imita al usuario haciendo clics).
- **JMeter:** Herramienta de la fundación Apache para pruebas de rendimiento, carga y estrés.
- **Cucumber:** Para pruebas basadas en BDD (Behavior Driven Development) usando lenguaje natural (Gherkin).

TDD (Test Driven Development): El ciclo Red-Green-Refactor

Dentro del bloque de pruebas y metodologías ágiles, esta técnica de diseño de software es recurrente:

- **Flujo (Red-Green-Refactor):** 1. **Red:** Escribir una prueba que falle.
2. **Green:** Escribir el código mínimo para que la prueba pase.
3. **Refactor:** Limpiar y mejorar el código escrito.



Pruebas de Regresión y Pruebas de Humo (Smoke Tests)

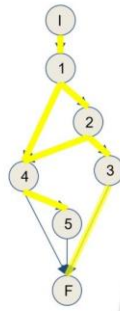
- **Pruebas de Regresión:** Se realizan después de modificar el software para asegurar que los cambios no han introducido errores en partes que antes funcionaban correctamente.
- **Pruebas de Humo:** Son pruebas rápidas para verificar que las funciones más básicas del sistema funcionan (ej. ¿el servidor arranca?). Si fallan, no se sigue probando.

Métricas de Calidad: La Complejidad Ciclomática de McCabe

Es la métrica técnica por excelencia en los exámenes de desarrollo.

- **Qué es:** Mide la complejidad lógica de un programa contando el número de caminos independientes a través del código fuente.
- **Fórmula:** $V(G) = E - N + 2P$ (donde E son aristas, N nodos y P componentes conexos).
- **Para qué sirve:** Para determinar el número mínimo de pruebas necesarias para cubrir todo el código.

```
public void aMethod() {  
    if (a && b) {  
        doSomething();  
    } else if (c) {  
        doSthElse();  
    }  
}
```



$$V(G) = \text{Aristas} - \text{Nodos} + 2$$

$$V(G) = 9 - 7 + 2 = 4$$

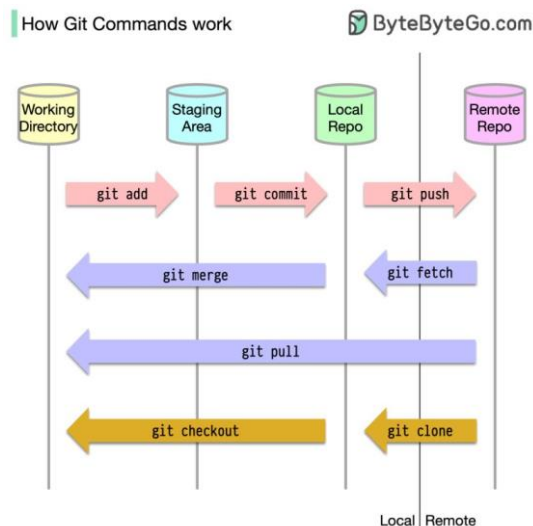
Programas para Control de Versiones

Clasificación

- **Centralizados:** Requieren un servidor central (ej. SVN - Apache Subversion).
- **Distribuidos:** Cada usuario tiene una copia completa (ej. Git, Mercurial).

Estados de los Archivos en Git

1. **Confirmado (committed):** Datos almacenados de manera segura en la base de datos local.
 2. **Modificado (modified):** El archivo se ha cambiado pero no se ha confirmado.
 3. **Preparado (staged):** Archivo modificado marcado para ir en la próxima confirmación.
- **Distractor:** "Retenido (holded)" NO es un estado de Git.



Comandos de Git

Comando	Función Literal
git commit	"Almacenar una nueva instantánea (snapshot) del estado del proyecto en el historial".
git push	"Subir commits de una rama local a repositorio remoto".
git merge	"Permitirá mezclar funcionalidades de ramas distintas".
git log	"Muestra una lista de los commits anteriores".
git tag	"Listar las etiquetas".
git rebase	Sirve para integrar cambios de una rama en otra, pero a diferencia del merge, reescribe el historial para que parezca lineal.
git stash	Permite guardar temporalmente los cambios que no están listos (en el "limbo") para poder cambiar de rama sin hacer commit.
git checkout vs git switch	El nuevo estándar en Git para cambiar de rama es git switch.
git cherry-pick	Es un comando "avanzado" que sirve para aplicar un commit específico de una rama a otra sin tener que hacer un merge completo.
git fetch	Descarga el historial y los cambios del repositorio remoto pero NO los aplica a tu rama de trabajo actual (solo actualiza el seguimiento remoto).
git pull	Es la combinación de git fetch + git merge. Descarga los cambios y los fusiona automáticamente en tu rama local.

Enfoque CPS: CPS pregunta por el comando exacto para subir cambios (push origin master) y por el nombre del fichero para ignorar archivos: .gitignore.

Plataformas de Desarrollo Colaborativo

- **GitHub / GitLab:** "Plataforma de desarrollo colaborativo para alojar proyectos utilizando el sistema de control de versiones Git... se puede almacenar, compartir y trabajar de forma conjunta en el código".

- **Jenkins:** Software que "permite automatizar la construcción y despliegue de un proyecto utilizando el concepto de pipelines".
- **SonarQube:** Herramienta para "análisis de seguridad y calidad de código". Es falso que sea una solución para análisis dinámico; es para análisis estático.
- **CI/CD:** Siglas de Continuous Integration / Continuous Delivery.
- **Fases del ciclo DevOps que forman un "infinito":**
 - **Fases:** Planificar -> Codificar -> Construir (Build) -> Probar -> Liberar (Release) -> Desplegar -> Operar -> Monitorizar.



- Es una de las trampas favoritas en las preguntas sobre DevOps.
 - **Continuous Delivery (Entrega Continua):** El código se prueba y construye automáticamente, pero el despliegue final a producción es manual (alguien pulsa un botón).
 - **Continuous Deployment (Despliegue Continuo):** Todo el proceso, incluido el paso a producción, es 100% automático. No hay intervención humana.
- **4 libertades del Software Libre (FSF):**
 1. **Libertad 0: Usar el programa para cualquier propósito.**
 2. **Libertad 1: Estudiar cómo funciona y adaptarlo (acceso al código).** requiere obligatoriamente el acceso al código fuente. Sin código fuente, no hay libertad 1 ni 3. Es una pregunta típica de "falso/verdadero".
 3. **Libertad 2: Distribuir copias.**
 4. **Libertad 3: Mejorar el programa y publicar las mejoras.**
- **Licencias:** Conocer la diferencia entre Copyleft (GPL - obliga a que las derivadas sean libres) y Permisivas (MIT/BSD - permiten que las derivadas sean cerradas).

Enfoque CPS Final: Memorice las herramientas que NO son de control de versiones, como JMeter (que es de pruebas) o Terraform (que es de infraestructura como código).

GitFlow: Es el nombre del "modelo de ramas" más famoso (Master, Develop, Feature, Release, Hotfix). A veces preguntan cómo se llama este flujo de trabajo.